CREATIONAL DESIGN PATTERNS

Course Name: Software Engineering Laboratory

Course Code: CSE402

Submission Date: 2026-03-11

Submitted by

Md. Salamoon Hossain

Student ID: 222248038

Batch: 222 **Section:** 1

Submitted to

Sheikh Hasanul Banna

Lecturer

Presidency University

Department of Computer Science & Engineering

Presidency University

Comparative Study of

Factory Pattern and Abstract Factory Pattern

1. Factory Pattern Explanation

The Factory Pattern provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created. In the provided `FactoryPattern` package:

- **Product Interface**: `Shape.java` defines the common behavior (`draw()`).
- **Concrete Products**: `Circle.java` and `Rectangle.java` implement the `Shape` interface.
- **Factory Class**: `ShapeFactory.java` contains the logic to instantiate specific shapes based on a String input.
- **Client**: `FactoryPatternDemo.java` interacts with the factory to obtain instances without knowing the specific implementation logic.

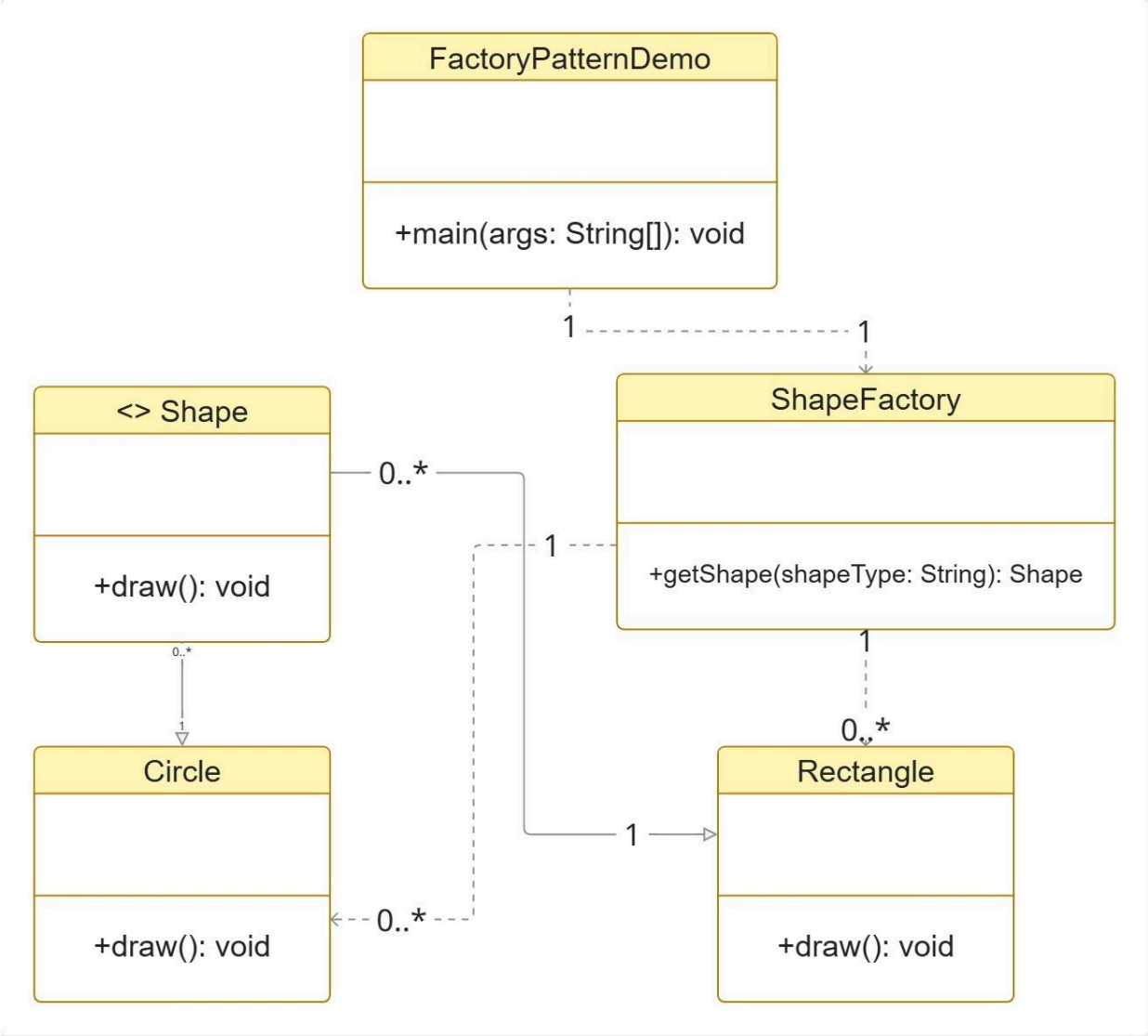
2. Abstract Factory Pattern Explanation

The Abstract Factory Pattern is a "factory of factories." It provides an interface for creating families of related or dependent objects without specifying their concrete classes. In the provided `AbstractFactory` package:

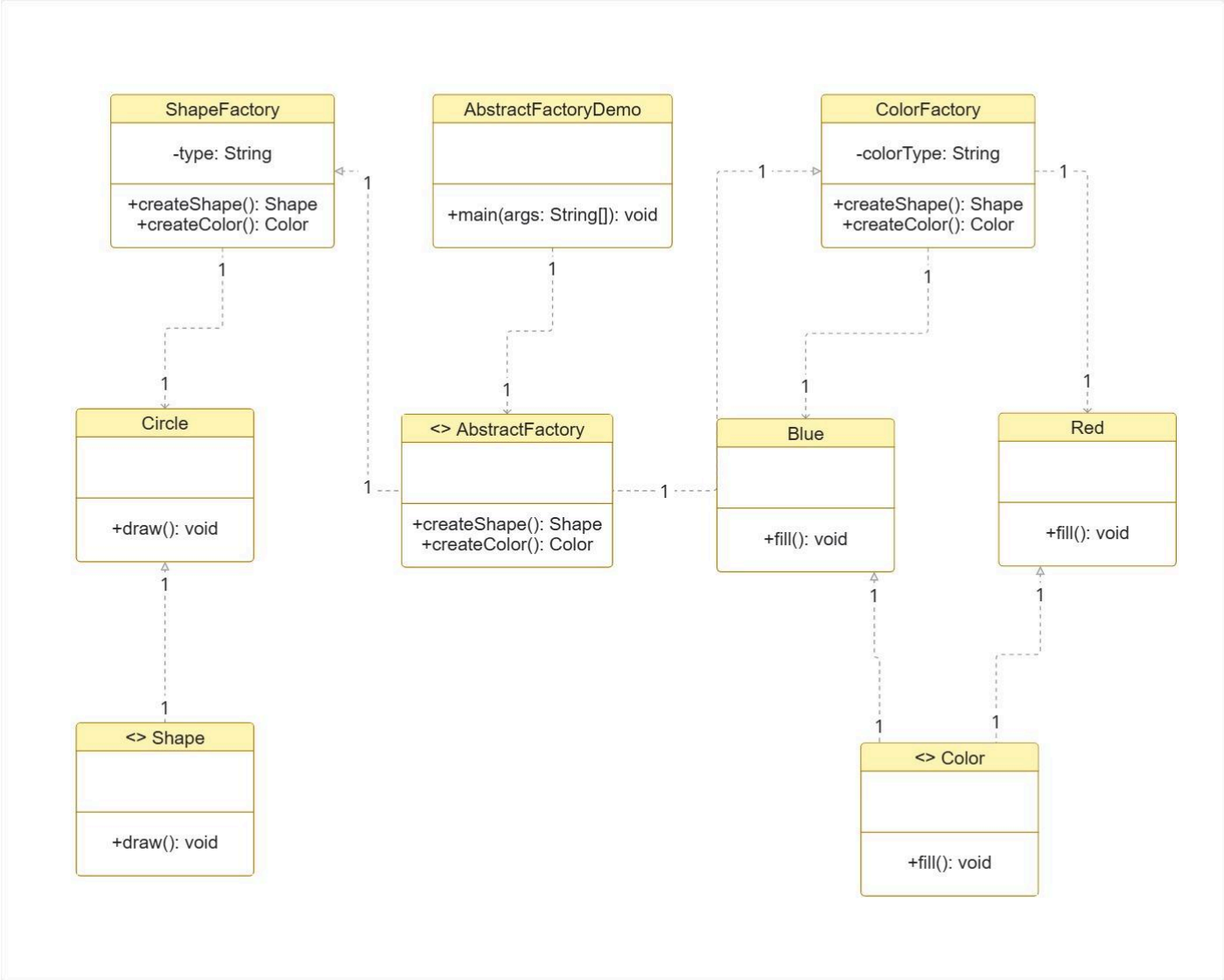
- **Abstract Factory**: `AbstractFactory.java` defines methods for creating both `Shape` and `Color`.
- **Concrete Factories**: `ShapeFactory.java` (in this package) and `ColorFactory.java` implement the creation logic for their respective families.
- **Abstract Products**: `Shape` and `Color` act as the base interfaces for the products.
- **Concrete Products**: `Red.java`, `Blue.java`, `Circle.java`, etc., provide the actual implementation.
- **Client**: `AbstractFactoryDemo.java` uses the factories to create themed objects (e.g., a specific shape and a specific color).

3. UML Diagram

Factory Pattern UML Diagram



Abstract Factory Pattern UML Diagram



4. Comparative Analysis

Feature	Factory Pattern	Abstract Factory Pattern
Complexity	Simple; involves one factory and one product hierarchy.	High; involves multiple factories and product families.
Object Creation	Creates objects through inheritance/method parameters.	Creates objects through composition of factory objects.
Scalability	Easy to add new product types (e.g., Square).	Difficult to add new product types to the interface (requires changing all factories).
Use Case	Use when the client doesn't know which concrete class it needs.	Use when your system must be independent of how its products are created and composed.

5. Code-Level Explanation

- **Client Interaction**: In the Factory Pattern, the client asks for a specific object using a string. In the Abstract Factory, the client first selects the type of factory it needs and then uses that factory to get products.
- **Polymorphism**: Both patterns utilize polymorphism by returning the interface type (`Shape` or `Color`) rather than the concrete class.

- **Coupling**: The Factory Pattern promotes loose coupling by hiding the `new` keyword from the client. The Abstract Factory further decouples the client by grouping related objects together.
- **Adding New Products**: In the provided `ShapeFactory`, adding a "Triangle" requires adding an `else if` block. In `AbstractFactory`, adding a new product category (like `Size`) would require modifying the `AbstractFactory` interface and all its implementations, violating the Open/Closed Principle.

6. Conclusion

The Factory Pattern is ideal for simple object creation where the specific type is determined at runtime. In contrast, the Abstract Factory Pattern is superior when a system must handle multiple families of related products, ensuring that products from different families are not mixed incorrectly.While the Abstract Factory introduces more architectural complexity and requires handling "not applicable" methods (returning `null`), it successfully ensures that the client remains decoupled from specific product implementations.